

Task 5: Capture and Analyze Network Traffic Using Wireshark:

Monitoring Network Traffic:

Analysis of HTTP, DNS, and TCP Protocols Using Wireshark:

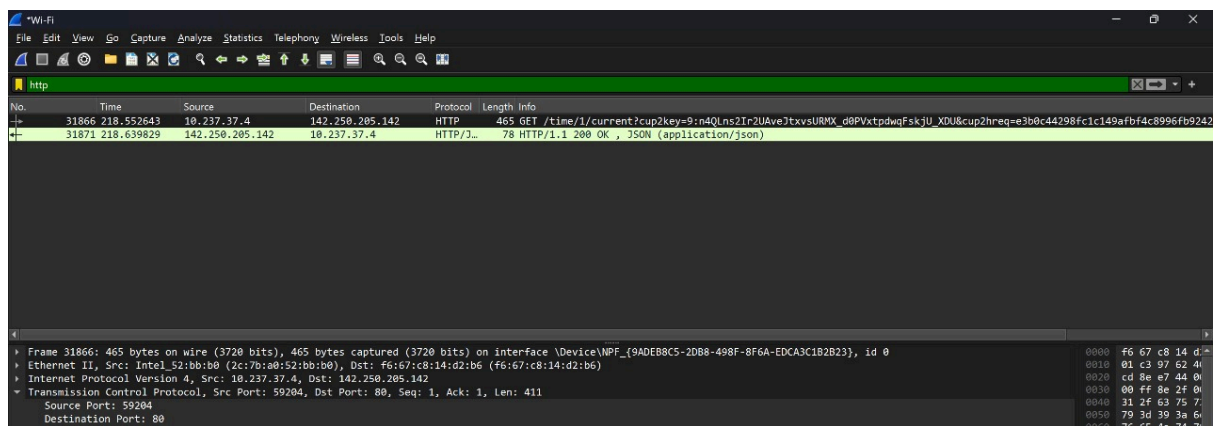
1. HTTP (HyperText Transfer Protocol):

- **Purpose:** Used for transferring web pages and data between a web browser (client) and a web server.
- **Observation:**

When accessing websites through Google Chrome, multiple HTTP request and response packets were captured.

These packets showed details like the request method (GET/POST), source and destination IP addresses, and response status codes.
- **Learning Outcome:**

Understood how web browsers communicate with servers using HTTP and how web content is transmitted across the network.



2. DNS (Domain Name System):

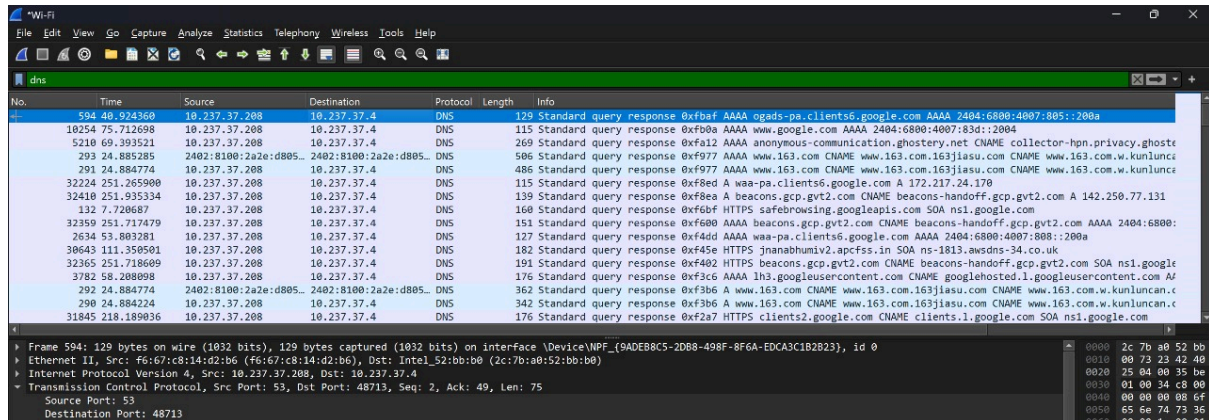
- **Purpose:** Translates human-readable domain names (like *www.google.com*) into IP addresses required for routing traffic.
- **Observation:**

DNS query and response packets were captured each time a new website was accessed. The packets contained information such as the

queried domain name, DNS server IP, and the resolved IP address.

- **Learning Outcome:**

Learned that DNS is the first step in establishing most network connections, enabling devices to find the correct destination servers.

A screenshot of the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The main display area is titled 'dns' and shows a list of captured packets. The packet list has columns for No., Time, Source, Destination, Protocol, Length, and Info. The selected packet (No. 129) is a DNS Standard query response from 10.237.37.208 to 10.237.37.4. The packet details pane on the right shows the structure of the DNS response, including the question section and the answer section with multiple resource records. The packet bytes pane at the bottom shows the raw data in hexadecimal and ASCII. The status bar at the bottom indicates the current packet is frame 594, 129 bytes on wire (1032 bits), 129 bytes captured (1032 bits) on interface 'DeviceNPF_{9ADE8B8C5-2DB8-498F-8F6A-EDCA3C1B2823}', id 0.

No.	Time	Source	Destination	Protocol	Length	Info
594	40.924360	10.237.37.208	10.237.37.4	DNS	129	Standard query response 0xfba4 AAAA ogads-ps.clients6.google.com AAAA 2404:6800:4007:805::200a
10254	75.712698	10.237.37.208	10.237.37.4	DNS	115	Standard query response 0xf00a AAAA www.google.com AAAA 2404:6800:4007:83d::2004
5210	69.393521	10.237.37.208	10.237.37.4	DNS	269	Standard query response 0xfa12 AAAA anonymous-communication.ghostery.net CNAME collector-hpn.privacy.ghoste
293	24.885285	2402:8100:2a2e:d805::	2402:8100:2a2e:d805::	DNS	586	Standard query response 0xf977 AAAA www.163.com CNAME www.163.com.163jiasu.com CNAME www.163.com.w.kunlunc
291	24.884774	10.237.37.208	10.237.37.4	DNS	486	Standard query response 0xf977 AAAA www.163.com CNAME www.163.com.163jiasu.com CNAME www.163.com.w.kunlunc
32224	251.265900	10.237.37.208	10.237.37.4	DNS	115	Standard query response 0xf8ed A waa-ps.clients6.google.com A 172.217.24.170
32410	251.935334	10.237.37.208	10.237.37.4	DNS	139	Standard query response 0xf8ea A beacons.gcp.gvt2.com CNAME beacons-handoff.gcp.gvt2.com A 142.250.77.131
132	7.728687	10.237.37.208	10.237.37.4	DNS	160	Standard query response 0xf6bf HTTPS safebrowsing.googleapis.com SOA ns1.google.com
32359	251.717479	10.237.37.208	10.237.37.4	DNS	151	Standard query response 0xf600 AAAA beacons.gcp.gvt2.com CNAME beacons-handoff.gcp.gvt2.com AAAA 2404:6800:
2634	53.803281	10.237.37.208	10.237.37.4	DNS	127	Standard query response 0xf4dd AAAA waa-ps.clients6.google.com AAAA 2404:6800:4007:808::200a
30643	111.358991	10.237.37.208	10.237.37.4	DNS	182	Standard query response 0xf45e HTTPS jianabumiv2.apcfss.in SOA ns-1813.awsdns-34.co.uk
32365	251.718609	10.237.37.208	10.237.37.4	DNS	191	Standard query response 0xf402 HTTPS beacons.gcp.gvt2.com CNAME beacons-handoff.gcp.gvt2.com SOA ns1.google
3782	58.288098	10.237.37.208	10.237.37.4	DNS	176	Standard query response 0xf3c6 AAAA lh3.googleusercontent.com CNAME googlehosted.l.googleusercontent.com AF
292	24.884774	2402:8100:2a2e:d805::	2402:8100:2a2e:d805::	DNS	362	Standard query response 0xf3b6 A www.163.com CNAME www.163.com.163jiasu.com CNAME www.163.com.w.kunluncan.c
290	24.884224	10.237.37.208	10.237.37.4	DNS	342	Standard query response 0xf3b6 A www.163.com CNAME www.163.com.163jiasu.com CNAME www.163.com.w.kunluncan.c
31845	218.189036	10.237.37.208	10.237.37.4	DNS	176	Standard query response 0xf2a7 HTTPS clients2.google.com CNAME clients.l.google.com SOA ns1.google.com

3. TCP (Transmission Control Protocol):

- **Purpose:** Provides reliable, ordered, and error-checked delivery of data between devices.
- **Observation:**

Captured several TCP packets showing the three-way handshake process (SYN, SYN-ACK, ACK) during connection setup.

Also observed data transfer and connection termination (FIN packets), confirming TCP's role in ensuring reliable communication.
- **Learning Outcome:**

Understood how TCP ensures data integrity and connection reliability for protocols like HTTP and HTTPS.

No.	Time	Source	Destination	Protocol	Length	Info
2	0.113049	10.237.37.4	3.228.156.114	TCP	55	54955 → 443 [ACK] Seq=1 Ack=1 Win=253 Len=1 (TCP segment of a reassembled PDU)
12	0.390546	3.228.156.114	10.237.37.4	TCP	70	443 → 54955 [ACK] Seq=1 Ack=2 Win=167 Len=0 SLE=1 SRE=2
32	4.092144	2402:8100:2a2e:d805	2600:1901:0:47fc::	TCP	86	40890 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1300 WS=256 SACK_PERM
34	4.100876	2402:8100:2a2e:d805	2402:8100:2a2e:d805	ICMPv6	134	Destination Unreachable (no route to destination)
44	4.400888	10.237.37.4	35.190.80.1	TCP	66	42910 → 443 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM
49	4.448695	35.190.80.1	10.237.37.4	TCP	70	443 → 42910 [SYN, ACK] Seq=0 Ack=1 Win=65535 Len=0 MSS=1320 SACK_PERM WS=256
50	4.448809	10.237.37.4	35.190.80.1	TCP	54	42910 → 443 [ACK] Seq=1 Ack=1 Win=65280 Len=0
51	4.449518	10.237.37.4	35.190.80.1	TLSv1.3	2076	Client Hello (SNL=a.nel.cloudflare.com)
54	4.504283	35.190.80.1	10.237.37.4	TCP	58	443 → 42910 [ACK] Seq=1 Ack=2023 Win=268032 Len=0
55	4.504283	35.190.80.1	10.237.37.4	TCP	58	443 → 42910 [ACK] Seq=1 Ack=1321 Win=268544 Len=0
56	4.523559	35.190.80.1	10.237.37.4	TLSv1.3	270	Server Hello, Change Cipher Spec, Application Data
57	4.524071	10.237.37.4	35.190.80.1	TLSv1.3	118	Change Cipher Spec, Application Data
59	4.548905	35.190.80.1	10.237.37.4	TLSv1.3	676	Application Data, Application Data
60	4.593739	10.237.37.4	35.190.80.1	TCP	54	42910 → 443 [ACK] Seq=2087 Ack=831 Win=64512 Len=0
66	6.522004	10.237.37.4	172.64.155.209	TLSv1.2	5476	Application Data
67	6.522146	10.237.37.4	172.64.155.209	TLSv1.2	93	Application Data

Frame 32: 86 bytes on wire (688 bits), 86 bytes captured (688 bits) on interface \Device\NPF_{9ADEB8C5-2DB8-498F-8F6A-EDCA3C1B2B23}, id 0
 Ethernet II, Src: Intel_52:bb:b0 (2c:7b:a0:52:bb:b0), Dst: f6:67:c8:14:d2:b6 (f6:67:c8:14:d2:b6)
 Internet Protocol Version 6, Src: 2402:8100:2a2e:d805:884:c28c:6ebd:396b, Dst: 2600:1901:0:47fc::
 Transmission Control Protocol, Src Port: 40890, Dst Port: 443, Seq: 0, Len: 0
 Source Port: 40890
 Destination Port: 443

Wireshark Analysis Summary:

Through Wireshark analysis, I learned how different protocols interact:

- **DNS** initiates communication by resolving domain names.
- **TCP** establishes a reliable connection between devices.
- **HTTP** transmits actual web data over the established TCP connection.

This exercise enhanced my practical understanding of **network packet structure, protocol flow, and the importance of monitoring network traffic** to identify normal and abnormal behavior.